RAW_Image 项目交接与开发准备指南

> 生成日期：2026-09-20 仓库地址：https://github.com/Hunter-Stuart/RAW_Image.git 本地路径：`D:\firstwork\RAW_Image` 适用读者：接手本项目的开发人员（需具备基础 Python / 图像处理知识）

目录

1. 项目内容获取方式（已解决）
2. 项目核心目标
3. 前期准备工作（环境 / 克隆 / 数据）
4. 论文与文献准备
5. 代码结构与运行调试
6. VS Code + Git 版本控制与工作日志
7. 已知问题与未完成模块
8. 附录：快速命令速查表

一、项目内容获取方式（已解决）

结论：采用"本地克隆 + 直接读代码"方式，已完成。

- 本机 Claude Code 已把仓库克隆到 `D:\firstwork\RAW_Image`（放在你的研究代码目录 `D:\firstwork` 下，与 3DGS、HDRSplat 等项目并列）。
- 不再需要 MCP 服务器或浏览器扩展；也不需要你手动粘贴文件。
- 本项目是公开仓库，克隆一次即可；若日后 GitHub 有更新，在仓库目录执行 `git pull` 即可同步。

> 注意：你本机直连 GitHub 会失败（Connection reset），必须走本地代理 `127.0.0.1:7897`（Clash Verge）。 建议一次性配置好，以后所有 git 操作都不用再管：

```
git config --global http.proxy  http://127.0.0.1:7897
git config --global https.proxy http://127.0.0.1:7897
# 若代理软件关闭导致 git 失败，可临时取消：
# git config --global --unset http.proxy && git config --global --unset
https.proxy
```

二、项目核心目标

2.1 一句话定位

> 本项目是**天府绛溪实验室 E-SIP 项目**中"传感组"的图像处理中间环节： **把双目相机的 RAW（Bayer）数据，处理成下游"双目 3D 目标检测"可以直接使用的 sRGB 图像**，并保证左右图像在时间、几何、亮度和色彩上的一致性。

整体数据流（来自项目 README）：

```
相机传感器（RAW）
    ↓
E-SIP ISP Pipeline（本仓库 binocular_isp 目录）
  RAW读取 → 黑电平/白平衡 → 去马赛克 → 色彩校正/Gamma
  → 去噪/锐化(可选) → 去畸变/极线校正 → 质量检查
    ↓
sRGB 图像（左右图 + 元数据 metadata.json）
    ↓
下游：双目立体匹配 → 视差/深度 → 3D 目标检测
```

2.2 下游任务对图像的具体要求（决定代码怎么写的关键约束）

来自仓库根目录 `README.md`：

维度	要求
格式	RGB888（8 bit/通道）、sRGB 色彩空间、无压缩 PNG（避免 JPEG）
分辨率 / 帧率	1280×720；≥ 20 FPS；端到端延迟 < 100 ms
时间同步	左右时间偏差 < 1 ms（动态场景控制在曝光时间内）
几何	左右内参/畸变/外参/基线/投影矩阵准确；极线校正后同名点垂直误差 < 0.5 px（最大 1 px）
动态范围	室外/逆光 70–80 dB；隧道口、强逆光希望达到 100 dB（HDR/WDR）
左右一致性	曝光、白平衡、Gamma、锐化尽量一致；禁止左右独立自动曝光
图像处理强度	不过度降噪/锐化/局部色调映射/几何裁切（立体匹配依赖边缘纹理）
元数据	时间戳、帧号、内参/畸变/外参、基线、校正矩阵、曝光/增益/WB/HDR 状态

2.3 技术架构

纯 Python + OpenCV 的**模块化传统 ISP 管线**（不是深度学习模型），按"每个 ISP 环节一个模块"组织：

```
src/
├── isp_pipeline/    # ISP 核心：raw_reader → pre_process → demosaic
│                    #         → color_correction → denoise_sharpen → pipeline(编排)
├── binocular/       # 双目：sync_control(左右一致性控制/检查)、rectify(极线校正)
├── calibration/     # 棋盘格单目/双目标定
├── config/          # YAML 配置加载与管理
└── utils/           # 元数据、可视化
```

2.4 当前进展阶段

- 仓库只有 2 个提交（`Initial commit` → `V0.0`），属于**初版（V0.0）阶段**。
- **已跑通**：模拟 Bayer 数据的完整管线（单元测试 4 项全部通过，本指南已实测）。
- **未完成**：
  - 相机标定参数是占位符（`configs/camera_*.yaml` 中的内参是假的）；
  - 主脚本中极线校正的外参（R/T）被硬编码为单位矩阵/零向量，**没有真正读取 `stereo_calibration.yaml`**（详见第七节）；
  - 后续计划（README 所列）：TA-ISP 式任务反馈闭环、HDR 融合、在线质量监测仪表板、ROS 接口、与 E-SIP Sensorkit 深度对接。

三、前期准备工作（环境 / 克隆 / 数据）

3.1 环境依赖

项目 `binocular_isp/requirements.txt` 声明：

库	版本要求	作用
numpy	≥1.24	数值计算
opencv-python	≥4.8	去马赛克、校正、滤波
rawpy	≥0.18	读取 RAW 文件（封装 LibRaw，**硬依赖**，无它连模拟测试都跑不了）
PyYAML	≥6.0	配置解析
tqdm / matplotlib	-	进度条 / 调试可视化
pytest	≥7.0	测试

- Python 版本建议 **3.10 ~ 3.12**（rawpy 在这些版本上 wheel 最全；你的 base 环境是 3.13，缺 rawpy）。
- 不需要 CUDA（当前阶段无深度学习模块）。

已验证的现成方案：你机器上已有 conda 环境 `adaptiveisp`（`D:\anaconda1\adaptiveisp`，Python 3.10.20）已装好 numpy 2.2.6 / opencv 5.0.0 / rawpy 0.27.0 / PyYAML / matplotlib，**直接可用**，只缺 pytest。本指南已用该环境实测跑通全部测试。

推荐做法（不污染现有环境）：新建专用环境：

```
conda create -n binocular_isp python=3.10 -y
conda activate binocular_isp
pip install -r D:/firstwork/RAW_Image/binocular_isp/requirements.txt
```

（如果图省事，也可以直接 `conda activate adaptiveisp` 然后 `pip install pytest`，实测可行。）

3.2 克隆项目（新机器上重复此步骤）

```
# 1) 配置代理（只需一次，见第一节）
git config --global http.proxy  http://127.0.0.1:7897
git config --global https.proxy http://127.0.0.1:7897
```

```
# 2) 克隆
git clone https://github.com/Hunter-Stuart/RAW_Image.git D:/firstwork/RAW_Image

# 3) 确认
cd D:/firstwork/RAW_Image && git log --oneline
```

3.3 数据集

- **当前阶段不需要下载任何数据集**：管线用 `simulate_bayer_raw()` 从普通 RGB 图模拟 Bayer 数据即可开发调试；仓库 `binocular_isp/data/` 里已有样例（`bayer_raw.png`、`gt_rgb.png`、`output_srgb.png`、合成棋盘格 20 张）。
- 真正需要的外部数据是**你自己采集的双目 RAW 图**（左右各一个目录，文件名一致），以及**真实双目标定结果**（用 `scripts/validate_calibration.py` 里的流程或 Kalibr/OpenCV 标定得到，再填入 `configs/camera_left.yaml`、`camera_right.yaml`、`stereo_calibration.yaml`）。
- 若做下游检测实验需要 RAW 检测数据集，可用 AODRaw 数据集（详见 4.3 节；作者提供百度网盘 / Google Drive / TeraBox 下载，含原图 6000×4000、降采样 2000×1333、切片 1280×1280 三种格式）。**注意：AODRaw 是单目检测数据集，与本项目双目场景不直接匹配，仅适合做 RAW 域算法对比实验。**

四、论文与文献准备

仓库自带的 `参考文献/` 目录已经替你整理好了一个**分级阅读计划**（约 120 篇 PDF，全部在仓库里），按此顺序读即可：

4.1 第一梯队：必读 3 篇（建立全局认知）

文件位置：`参考文献/01-必读-全局认知/`

顺序	论文	读后应能回答
①	Efficient visual computing with camera RAW snapshots（综述）	RAW 和 sRGB 有什么区别？为什么要用 RAW 做视觉？
②	Raw or cooked? Object detection on RAW images	直接用 RAW 检测 vs 先 ISP 再检测，各有何优劣？
③	Overview of ISP parameter tuning for visual perception	ISP 各模块是干什么的？参数对下游任务影响多大？

4.2 第二梯队：核心 3 篇（理解"任务驱动 ISP"）

文件位置：`参考文献/02-核心-任务驱动ISP/`

顺序	论文	对应关系
④	Task-aware ISP for advanced visual perception（即 TA-ISP，CVPR 2026）	本项目"闭环反馈"阶段的核心参考方向

顺序	论文	对应关系
⑤	DynamicISP: dynamically controlled ISP	ISP 参数如何按图像区域/内容自适应
⑥	Dirty pixels: towards end-to-end image processing	端到端 ISP 与感知联合学习的思路

4.3 两篇关键外部论文（项目 README 明确点名）

AODRaw（CVPR 2025 Highlight）

- 论文："Towards RAW Object Detection in Diverse Conditions"，arXiv:2411.15678
- 代码：https://github.com/lzyhha/AODRaw ；配套 mmdetection：https://github.com/lzyhha/AODRaw-mmdetection
- 内容：7,785 张高分辨率真实 RAW（Sony A7M4，6000×4000）、62 类、135,601 个实例、9 种光照×天气组合（白天/暗光 × 晴/雨/雾）。核心发现：RAW 域预训练 + 跨域知识蒸馏可超过 sRGB 检测。
- **与本项目代码的对应关系**：`raw_reader.py`（RAW 读取与元数据提取）参考了 AODRaw 的预处理代码；管线稳定后，可对比"RAW 域直接检测 vs 本项目 ISP 后检测"。

TA-ISP（CVPR 2026，电子科技大学 CVL 实验室）

- 论文："Task-Aware Image Signal Processor for Advanced Visual Perception"，arXiv:2509.13762
- 代码：https://github.com/CVL-UESTC/TA-ISP
- 内容：仅 ~3K 参数的轻量任务感知 ISP，用全局亮度校准 + 层级空间注意力 + 区域引导特征调制替代传统 ISP 模块，与下游任务端到端联合优化。
- **与本项目代码的对应关系**：是 `binocular_isp` README"后续计划"中**任务反馈闭环**（根据下游检测结果自动调整 ISP 参数）的参考实现。

4.4 按需查阅：12 个专题

`参考文献/03-按需查阅/` 下的专题与本项目模块的直接对应关系（开发到哪个模块就查哪个专题）：

专题目录	对应本项目模块
01-RAW目标检测	整体方向（下游检测）
02-ISP设计与调优	`isp_pipeline/` 各模块参数选择
03-端到端学习ISP	远期路线（学习型 ISP）
04-自适应与可重构ISP	`sync_control.py` + 后续闭环
05-低照度与暗光增强	`denoise_sharpen.py`、HDR 规划
06-曝光控制(AE-AGC)	`sync_control.py` 的曝光一致性
07-Bayer域特征提取	`raw_reader.py` 的 Bayer 操作
08-SLAM与视觉里程计	`rectify.py` 的鲁棒性要求
09-光度标定与成像模型	`calibration/`、黑电平/白平衡

专题目录	对应本项目模块
10-3DGS-HDR与新视角合成	HDR 融合规划（与你 HDRSplat 项目呼应）
11-In-Sensor与边缘计算	E-SIP 大纲第 6 部分（远期）
12-其他与综合	扩展阅读

五、代码结构与运行调试

5.1 核心文件清单（按"阅读顺序"排列，这是你上手代码的顺序）

第一步：先读入口脚本

文件	作用
`binocular_isp/scripts/run_pipeline.py`	**主入口**：单对/批量处理双目 RAW → sRGB，含校正与一致性检查
`binocular_isp/scripts/batch_process.py`	批量处理（简化版，不做校正）
`binocular_isp/scripts/validate_calibration.py`	标定数学"开卷考试"（合成棋盘格验证标定精度）

第二步：读管线核心（数据流向）

文件	作用
`src/isp_pipeline/pipeline.py`	**编排器 `ISPPipeline`**：把下面 5 个模块串起来，负责缩放、位深转换
`src/isp_pipeline/raw_reader.py`	RAW 读取（rawpy）、元数据提取、Bayer 通道分离、`simulate_bayer_raw` 模拟数据
`src/isp_pipeline/pre_process.py`	黑电平校正、白平衡（支持灰世界自动估计）、归一化
`src/isp_pipeline/demosaic.py`	去马赛克（opencv / bilinear / malvar 三种）
`src/isp_pipeline/color_correction.py`	色彩矩阵、sRGB 转换、Gamma（含 sRGB 标准分段式）、色调映射
`src/isp_pipeline/denoise_sharpen.py`	去噪（bilateral/nlm/median/gaussian）与锐化（unsharp/laplacian），默认关闭

第三步：读双目模块

文件	作用
`src/binocular/sync_control.py`	左右曝光/WB 参数统一（`StereoConsistencyController`）+ 左右亮度/颜色/清晰度一致性检测（`StereoConsistencyChecker`）
`src/binocular/rectify.py`	`StereoCalibration` 数据结构 + `StereoRectifier`（stereoRectify 极线校正 + ORB 匹配验证垂直误差）

文件	作用
src/calibration/calibrator.py	棋盘格单目/双目标定（MonoCalibrator / StereoCalibrator）

第四步：读配置与工具

文件	作用
src/config/camera_params.py	ConfigLoader（YAML 加载）+ 各默认配置模板
src/config/isp_config.py	ISPConfig 类（点号路径读写 YAML）
src/utils/metadata.py	输出元数据管理（metadata.json）
src/utils/visualization.py	调试可视化
configs/pipeline_config.yaml	ISP 管线参数（黑电平、白平衡、Gamma、去噪等）
configs/camera_left.yaml / camera_right.yaml	左右相机内参（**当前是占位符**）
configs/stereo_calibration.yaml	双目标定外参（**当前是占位符**）
tests/test_pipeline.py	模拟数据单元测试（4 项）

5.2 入口点

- **学习/调试入口**：tests/test_pipeline.py —— 不依赖真实 RAW，直接跑通全链路。
- **业务入口**：scripts/run_pipeline.py —— 处理真实双目 RAW。

5.3 最小可行运行步骤（已实测验证）

```
# 0) 激活环境（本机已验证可用）
conda activate adaptiveisp
pip install pytest          # 该环境唯一缺的包，装一次即可

# 1) 进入项目
cd D:/firstwork/RAW_Image/binocular_isp

# 2) 跑单元测试（模拟 Bayer 数据走完整管线，实测 4/4 通过）
python tests/test_pipeline.py

# 3) 跑标定开卷考试（验证标定数学，实测通过，误差 <0.01px）
python scripts/validate_calibration.py

# 4) 处理真实双目 RAW（需要先做两件事，见下方说明）
python scripts/run_pipeline.py \
    --left  data/input/left_0001.ARW \
    --right data/input/right_0001.ARW \
    --output data/output \
    --rectify \
    --check-consistency
```

第 4 步前需要：

1. 把真实 RAW 放到 `data/input/` 左右两个目录（文件名要一一对应）；
2. 把真实标定值填入 `configs/camera_left.yaml`、`camera_right.yaml`（内参、畸变）——**注意**：当前脚本极线校正用的 R/T 是硬编码的单位矩阵/零向量，若要真正做双目校正，需要先修 `run_pipeline.py` 第 171-172 行让它读 `stereo_calibration.yaml`（见第七节）。

没有真实 RAW 时，可以用 `raw_reader.simulate_bayer_raw()` 从任意 RGB 图生成 Bayer 数据做调试（测试文件里就是这么干的）。

六、VS Code + Git 版本控制与工作日志

> 原则：**小步提交、每次修改前先有稳定快照、日志跟着提交走、push 前三思。**
>
> ⚠️ 特别提醒：这是**团队共用的实习仓库**（同事也在用），你的每次 `git push` 都会影响所有人。日常开发务必在**自己的分支**上进行，合并回 `main` 前先沟通确认。

6.1 第一次：建立基线快照（只做一次）

```
cd D:/firstwork/RAW_Image
git tag v0.0-baseline          # 给当前 V0.0 打一个本地标签，作为"初始稳定快照"
git log --oneline              # 确认：3b08fe9 (V0.0) 是当前状态
```

以后任何时刻想回到这个初始状态：`git checkout v0.0-baseline`。

6.2 每次动手前：创建快照的三种方式（按改动大小选择）

方式 A：小改动（改一个文件、调参数）——直接提交即可

```
git status                          # 先看改了什么
git add <具体文件>                   # 只添加相关文件，不要 git add . 一把梭
git commit -m "fix: 修复白平衡增益计算中除零风险"
```

方式 B：中等改动（半天以内的活）——开功能分支

```
git checkout -b feature/hdr-fusion    # 从当前状态开新分支
# ...开发、小步提交...
git checkout main                     # 做完回到 main
git merge --no-ff feature/hdr-fusion  # 合并（保留分支历史）
```

方式 C：大改动且前途未卜——用 `git stash` 临时保存 + 分支兜底

```
git checkout -b experiment/xxx   # 先开分支再动手，永远别在 main 上直接改
# 若中途想"先放下手头实验去修个 bug":
git stash push -m "实验：色彩矩阵改用 rawpy 的 rgb_camera"
# ...去修 bug...
git stash pop                    # 回来继续
```

6.3 工作日志与提交关联（推荐约定）

约定一：提交信息用统一格式（VS Code 里写多行提交信息很方便）：

```
<类型>: <一句话概括改了什么>

原因: <为什么改 / 想解决什么问题>
影响: <影响了哪些功能，如何验证>
```

类型用 `feat`(新功能) / `fix`(修复) / `refactor`(重构) / `docs`(文档) / `test`(测试) / `chore`(杂项)。

约定二：维护一个工作日志文件 `WORKLOG.md`（放在仓库根目录，每行一条）：

```
## 2026-09-20
- 在 feature/xxx 分支修复白平衡除零风险（提交 8f3a2c1），原因：灰世界估计在纯黑图上会除零。
- 学习 TA-ISP 论文，梳理闭环反馈方案（无代码改动，笔记见 docs/ta-isp-notes.md）。
```

每天收工前花 1 分钟补一行日志。日志与提交互相印证：**提交记录回答"代码怎么变的"，日志回答"为什么这么变、明天从哪继续"**。日志文件本身也提交（类型 `docs`），这样远程仓库里日志和代码永远同步。

6.4 在 VS Code 中查看历史与回滚（图形界面操作）

想做什么	VS Code 操作
看某文件的历史	资源管理器中右键文件 → **打开时间线**（Timeline），或左侧 **源代码管理** 面板看未提交改动
看整体提交历史	安装扩展 **Git Graph**（或 GitLens）：点左侧图标 → 显示分支/提交树，点击任意提交可看完整 diff
对比任意两个版本	右键文件 → **与已提交版本比较** / 选择任意提交比较
回滚某个文件	文件历史里右键某次提交 → **还原**(Revert)，或命令行 `git checkout <提交号> -- <文件>`
回滚整个项目到某次提交	命令行 `git checkout v0.0-baseline`（只读查看）或 `git revert <提交号>`（生成一个"反向提交"，**推荐**，历史可追溯）；`git reset --hard` 会抹掉提交历史，**除非确认不要了否则别用**
撤销还没提交的改动	源代码管理面板：文件上点 **放弃更改**（Discard）；命令行 `git stash` 更安全（改动还在 stash 里）

6.5 与 GitHub 远程同步

```
# 开工前：拉取同事的最新代码（rebase 让历史保持线性）
git checkout main
git pull --rebase origin main

# 收工后：推送自己的分支/日志
git push origin main              # ⚠️ push 前先确认：这会影响所有同事！
git push origin <你的分支名>      # 推个人分支则完全安全
```

建议的日常节奏：

1. 开工：`git pull --rebase` → 看 `git log` 了解同事改了什么 → 在 `WORKLOG.md` 里补一行"今日计划"。
2. 开发：在自己的分支上小步提交（每完成一个可验证的小功能就提交一次）。
3. 收工：更新 `WORKLOG.md` 并提交 → 推送到远程 → 确认 `git status` 干净。

6.6 顺手建议：补一个 .gitignore

仓库目前没有 `.gitignore`（`__pycache__` 都被提交进仓库了）。建议在根目录新建：

```
__pycache__/
*.pyc
.pytest_cache/
binocular_isp/data/output/
binocular_isp/data/input/
*.log
```

创建后提交一次即可（已有的 `__pycache__` 可用 `git rm -r --cached binocular_isp/src --dry-run` 预览清理）。

七、已知问题与未完成模块

接手后**优先处理**的事项（按优先级排序）：

1. **【高】极线校正外参未接入**：`run_pipeline.py` 第 171-172 行把 R 硬编码为单位矩阵、T 为零向量，`stereo_calibration.yaml` 里的外参从未被读取。填了标定值也不会生效，必须修改脚本接入外参后才能做真正的双目校正。
2. **【高】相机标定配置是占位符**：三个 YAML 里的内参（fx=600 等）是假数据，必须用真实标定值替换（`validate_calibration.py` 已证明标定代码本身可用）。
3. **【中】BGR/RGB 通道顺序隐患**：`demosaic.py` 用 `COLOR_BAYER_RGGB2BGR` 输出的是 BGR 顺序，但 `pipeline.py` 后续按 RGB 处理、`cv2.imwrite` 直接写入，红蓝通道可能颠倒。现有测试只查形状不查颜色，没暴露此问题。建议对照样例图 `data/gt_rgb.png` 用红/蓝纯色模拟数据实测一次。
4. **【中】色彩校正矩阵链路需核实**：`process_file()` 把 rawpy 的 `rgb_camera` 矩阵（官方定义是 camera→sRGB）当作色彩矩阵传入，而 `convert_to_srgb()` 还会再乘一次 XYZ→sRGB 矩阵，存在"双重转换"嫌疑。建议对照 rawpy 文档与 AODRaw 预处理代码核实。
5. **【低】小项**：`batch_process.py` 未接校正/一致性检查；测试覆盖少；无 .gitignore；`data/output_srgb.png` 等结果文件混在仓库里；`camera_left.yaml` 带 BOM 头（不影响 PyYAML，但建议去掉）。

未完成模块（README 已列出的后续计划）：

- ☐ TA-ISP 风格任务反馈闭环（下一步最有价值的方向，论文见 4.3）
- ☐ HDR 融合支持（对应下游 100 dB 动态范围要求）
- ☐ 在线质量监测仪表板
- ☐ ROS 接口集成
- ☐ 与 E-SIP Sensorkit 深度对接

附录：快速命令速查表

```
# ── 环境 ──
conda activate adaptiveisp                          # 本机已验证可用的环境
conda create -n binocular_isp python=3.10 -y        # 或新建专用环境
pip install -r D:/firstwork/RAW_Image/binocular_isp/requirements.txt

# ── 测试 ──
cd D:/firstwork/RAW_Image/binocular_isp
python tests/test_pipeline.py                       # 单元测试（4项）
python scripts/validate_calibration.py              # 标定开卷考试

# ── 处理真实数据 ──
python scripts/run_pipeline.py --left <左RAW> --right <右RAW> \
    --output data/output --rectify --check-consistency
python scripts/batch_process.py --left-dir data/input/left \
    --right-dir data/input/right --output data/output
```

```
# ── Git 快照 / 回滚 ──
git tag v0.0-baseline                              # 初始基线（一次性）
git checkout -b feature/xxx                        # 开分支再动手
git add <文件> && git commit -m "..."              # 小步提交
git stash push -m "..." / git stash pop            # 临时保存/恢复
git checkout <提交号> -- <文件>                    # 单文件回滚
git revert <提交号>                                # 安全回滚（保留历史）
git pull --rebase origin main                      # 开工前同步
git push origin main                               # ⚠️ 团队仓库，push 前确认

# ── 网络 ──
git config --global http.proxy http://127.0.0.1:7897   # 一次性配置代理
```

本指南由 Claude Code 基于对仓库的完整阅读（全部源码、配置、参考文献目录与参考资料）生成，环境与测试结论均在本机实测验证。